

Synapse vs. Graphify vs. No Knowledge Graph — Accuracy Comparison

Date: 2026-09-18 **Purpose:** three-way accuracy comparison for presentation use: **No Graphify** (no knowledge graph at all) vs. **Existing Graphify** (the unmodified upstream engine) vs. **Synapse** (this project's engine + VS Code extension, built on top of graphify).

This report is a companion to [ACCURACY_REPORT.md](#) (the original 2-way graphify-vs-baseline study). This one adds the third arm — the *pre-enhancement* graphify engine — and evaluates it against the version of the engine Synapse ships today.

0. What "Synapse" means here, and one thing that had to be fixed first

Three engine features — **weighted retrieval**, **community-aware retrieval**, and **semantic graph** — were developed by a contributor (Charuvarthan-T) and pushed to three separate remote branches (`feature/weighted-retrieval`, `feature/community-retrieval`, `feature/semantic-graph`). **They had not been merged into `main`** . The modules existed in the repository's remote but `main`'s `graphify query / graphify path` never imported or called them — confirmed by checking that no file on `main` referenced `weighted_retrieval.py`, `community_retrieval.py`, or `semantic_graph.py` .

As part of preparing this comparison, the three branches were merged into `main` (three sequential merges, 0 conflicts, done 2026-09-18). Post-merge:

- **Weighted retrieval** — default-**on** in `graphify query / graphify path` (opt out with `--unweighted`)
- **Community-aware retrieval** — opt-in via `--community-aware`
- **Semantic graph** — a separate opt-in step, `graphify semantic-graph` (LLM-based; smoke-tested live against Groq in this session: **+6 edges, 2 edges augmented** on a 20-node sample)

Regression check after merging: full test suite = 3732 passed, 42 failed, 46 skipped. All 42 failures were re-run against the pre-merge codebase in an isolated worktree — 41 fail identically there (pre-existing Windows-only issues: symlink handling, long paths, `.gitignore` edge cases, unrelated to the 3 merged features). The 1 remaining (`" test_codex_hook_command_is_a_real_cli_subcommand "`) is a false positive from this machine's project folder containing spaces (`B Tech` , `7th Sem`) breaking the test's naive command-string parsing — confirmed identical on both old and new code when run from a space-free path. **Net new regressions from the merge: 0.**

"Synapse" in the tables below = `main` after this merge, i.e. what the VS Code extension now actually ships with.

1. Headline numbers

Retrieval accuracy (graph traversal, no LLM)

Method	Precision@10	Recall@10	MRR
No Graphify (naive grep keyword search)	0.083	0.75	0.318
Existing Graphify (pre-merge engine, plain BFS query)	0.050	0.45	0.251
Synapse (weighted + community-aware query)	0.075	0.70	0.271

CrossCodeEval code-completion accuracy (real LLM, Groq qwen/qwen3.8-27b, n=47 tasks)

Condition	Exact Match	Edit Similarity
No Graphify (no cross-file context)	6.4%	0.533
BM25 context (generic RAG baseline, for reference)	12.8%	0.595
Synapse context (weighted + community-aware)	29.8%	0.689

(Existing Graphify's full-47 number could not be completed — see §3 — but a fair paired n=19 comparison below shows the same ordering.)

Paired comparison, same 19 tasks, all four conditions

Condition	Exact Match	Edit Similarity
No Graphify	0.0%	0.433
BM25 context	15.8%	0.637
Existing Graphify	21.1%	0.672
Synapse	31.6%	0.734

Bottom line: Synapse's added retrieval features (weighted traversal + community-aware search) measurably improve on the pre-merge graphify engine — roughly **+10 points of exact-match accuracy** on real code completion (21.1% -> 31.6% on the paired sample; 25.5%¹ -> 29.8% on the full

historical-vs-fresh comparison) — and both graphify variants comfortably beat having no knowledge graph at all on the downstream LLM task. On the narrower graph-traversal-only metric (§1, top table), plain grep still edges out both graphify variants on precision/recall at this small single-repo scale — consistent with the original report's finding that graphify's traversal is built to hand an LLM *broad supporting context*, not to be a precision top-k search engine by itself.

2. Retrieval accuracy — methodology

Corpus: the same 7 files as the original report (`build.py` , `serve.py` , `security.py` , `export.py` , `benchmark.py` , `paths.py` , `__main__.py` from `graphify/`), frozen into an isolated corpus directory and extracted fresh with `graphify extract --code-only` (pure AST/tree-sitter, no LLM, no API key). Both engine versions produced an **identical 259-node / 536-edge graph** from this corpus — confirming the 3 merged features change *retrieval/query-time behavior only*, not AST extraction.

20 hand-written questions with known-correct targets (`tools/eval/ground_truth.json` , unchanged from the original study). Each engine's `graphify query "<question>"` was run against its own build of the graph:

- **Existing Graphify:** an isolated `git worktree` checked out at the pre-merge commit (`5db7e4e`), its own `uv` -sync'd environment, plain query (no `--weighted` / `--community-aware` flags exist on this commit).
- **Synapse:** current `main` , `graphify query "<question>" --community-aware` (weighted is already default-on).
- **No Graphify:** the same naive grep/keyword baseline from the original harness (scores `def` / `class` lines by question-term overlap) — no engine involved at all.

Scored with Precision@10 / Recall@10 / MRR against the same ground truth.

3. CrossCodeEval — methodology and an honest data gap

Dataset: [CrossCodeEval Python](#), the same 47 tasks as the original report — 23 from [nccgroup/libslub @ 7732a54](#) , 24 from [vladkens/twscrape @ 4f2ee7f](#) , both freshly re-cloned at the exact pinned commits and verified against the dataset's `prompt` field.

For each task, three (or four) completions were generated with `qwen/qwen3.8-27b` via Groq's free-tier OpenAI-compatible endpoint and scored with Exact Match + `difflib` edit similarity against the gold line:

1. **no_context** — bare in-file prefix
2. **bm25_context** — the dataset's own precomputed BM25 cross-file chunks
3. **graphify_context (Existing Graphify)** — `graphify query` run through the isolated pre-merge worktree's own compiled CLI against a fresh AST-only graph of that repo/commit
4. **graphify_context (Synapse)** — the same, but through the merged, current engine with `--community-aware`

Data gap, disclosed: Groq's free tier enforces an account-wide **200,000-tokens-per-day** cap. This session's cumulative usage (diagnostics, the retrieval harness's LLM-free calls, the semantic-graph smoke test, and the full Synapse run's 141 calls) consumed nearly all of it before the Existing-Graphify run reached its later tasks — **28 of 47 tasks in that run hit `429 rate_limit_exceeded` and returned no completion**; a retry after backing off still hit the same wall (the quota was not recovering meaningfully within the session). Rather than paper over this, the 19 tasks that *did* complete are reported as a real n=19, and a **paired subset comparison across all four conditions on those same 19 task IDs** is given in §1 so the comparison stays apples-to-apples. The full-47 numbers for No Graphify / BM25 / Synapse (which completed before the quota was exhausted) are also reported since they're the more statistically solid of the two tables.

To get a complete, uncontaminated n=47 for Existing Graphify: re-run tomorrow (quota resets) or with a paid Groq tier, using the exact command in §6.

4. Feature-by-feature comparison

Capability	No Graphify	Existing Graphify	Synapse
Structural code graph (AST, no LLM)	No	Yes	Yes (identical extractor)
Graph query / traversal	No (grep only)	Yes — BFS/DFS, unweighted	Yes — BFS/DFS, weighted by default
Relationship-aware ranking (call/import/inherit weighted differently)	No	No	Yes (<code>weighted_retrieval.py</code> , <code>weights.py</code>)
Community-aware search (restrict to relevant subgraph first)	No	No	Yes — opt-in <code>--community-aware</code>
Semantic (LLM-inferred) relationship edges	No	No	Yes — opt-in <code>graphify semantic-graph</code>
GitHub Copilot Chat integration (Language Model Tool)	No	No (CLI only)	Yes — <code>synapse_query / synapse_explain</code>
Dashboard / sidebar UI, onboarding walkthrough	No	No	Yes (VS Code extension)

Capability	No Graphify	Existing Graphify	Synapse
Needs an API key to build the graph	—	No	No (semantic graph/community naming are opt-in extras that do)

5. VS Code extension (the Synapse UI layer)

- Unit tests: **9/9 passing** (`npm run test:unit`)
- TypeScript compile: **clean**, no errors
- Electron integration suite (4 tests, requires downloading a VS Code test binary) was not re-run in this sandboxed session — no reason to expect a change, since this session touched the Python engine, not the extension.

6. Reproducing this report

```
# Retrieval accuracy – no API key needed
uv run graphify extract <corpus_dir> --code-only --no-cluster
uv run graphify query "<question>" --graph <corpus_dir>/graphify-out/graph.json --community-aware

# CrossCodeEval – needs a Groq (or any OpenAI-compatible) key
export OPENAI_API_KEY=<groq-key>
export OPENAI_BASE_URL=https://api.groq.com/openai/v1
# clone nccgroup/libslub@7732a54 and vladkens/twsrape@4f2ee7f, extract --code-only in each,
# download the CrossCodeEval Python parquet, then score no_context / bm25_context / graphify_context
# with qwen/qwen3.8-27b as in tools/eval/crosscodeeval.py (this report used a lightly adapted
# version that can point at either engine build and pass extra `graphify query` flags).

# Existing-Graphify-only comparison – build a git worktree at the pre-feature commit:
git worktree add -d <dir> 5db7e4e && cd <dir> && uv sync
```

7. Limitations

- Small samples: 20 retrieval questions, 47 (19 for one arm) completion tasks — directional, not a publishable-scale claim. Matches the scope of the original `ACCURACY_REPORT.md`.
- Free-tier model (`qwen/qwen3.8-27b`) — not frontier-tier; results may differ with a stronger model.
- Existing-Graphify CrossCodeEval arm is n=19, not n=47, due to the Groq daily token cap being exhausted mid-session (see §3). The paired comparison mitigates this but a full-scale re-run is recommended before treating the exact percentages as final.
- Retrieval ground truth is hand-labeled/self-authored (unchanged from the original study).

- Node/edge counts in the retrieval corpus (259/536) differ slightly from the original report's (257/448) because `serve.py / cli.py` have grown since that report was written — expected, not an error.
-

Appendix: raw result files

- `tools/eval/retrieval_results.json`, `tools/eval/ground_truth.json` — original 2-way study (kept for history)
 - `tools/eval/retrieval_results_existing_graphify.json` — this report's Existing Graphify retrieval run
 - `tools/eval/retrieval_results_synapse.json` — this report's Synapse retrieval run
 - `tools/eval/crosscodeeval_results_synapse.json` — full n=47 Synapse CrossCodeEval run
 - `tools/eval/crosscodeeval_results_existing_graphify_partial.json` — n=19 Existing Graphify CrossCodeEval run (see §3 for why it's partial)
-

1. 25.5% is the original `ACCURACY_REPORT.md` figure, measured before this session's merge and before some intervening `serve.py / cli.py` changes; the 29.8% figure above is a same-day fresh run of the *current* Synapse engine on the identical dataset/model, for a clean before/after. ■